

# LLMBA: Efficient Behavior Analytics via Large Pretrained Models in Zero Trust Networks

Senming Yan<sup>1</sup>, Graduate Student Member, IEEE, Lei Shi<sup>2</sup>, Graduate Student Member, IEEE,  
Wei Wang<sup>3</sup>, Senior Member, IEEE, Jing Ren<sup>4</sup>, Member, IEEE, Ying Li, and Limin Sun<sup>5</sup>

**Abstract**—Guided by the principle of “Never Trust, Always Verify”, Zero Trust Architecture (ZTA) mandates continuous monitoring and analysis of users and entities, highlighting the critical role of behavior analytics. However, the growing volume of audit data and its complex contextual information render many existing behavior analytics methods insufficient. Moreover, most approaches rely on high-quality labeled data for supervised training, limiting their effectiveness against previously unseen malicious behaviors. To address these challenges, we propose the Large Language Model for Behavior Analytics (LLMBA) framework. LLMBA leverages a Large Language Model (LLM) to analyze behavioral patterns of internal users and entities, capitalizing on the LLM’s strong ability to model sequential data. We introduce a multi-level behavior encoding scheme to capture both contextual and temporal information from behavior records, producing rich input representations for the LLM-enhanced model. The LLM is fine-tuned using self-supervised learning, enabling the detection of unknown malicious behaviors. To reduce the computational and storage overhead inherent in LLMs, we apply knowledge distillation to compress the model while maintaining high detection performance. Extensive experiments on the CERT Insider Threat dataset demonstrate that LLMBA outperforms state-of-the-art baselines in detection accuracy. Furthermore, the compressed student model achieves superior performance compared with existing methods under comparable runtime constraints, making LLMBA highly suitable for real-world deployment.

**Index Terms**—Cyber security, behavior analytics, threat detection, large language models, knowledge distillation.

Received 9 July 2025; revised 19 January 2026; accepted 12 February 2026. Date of publication 19 February 2026; date of current version 4 March 2026. This work was supported in part by the Major Key Project of Pengcheng Laboratory under Grant PCL2024A03, in part by the National Natural Science Foundation of China under Grant 92467201 and Grant U20A20156, in part by the Key Research Project of Chinese Academy of Sciences under Grant RCJJ-145-24-17, and in part by Beijing Natural Science Foundation under L234033. An earlier version of this paper was presented at the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP’26). The associate editor coordinating the review of this article and approving it for publication was Dr. Jason (Minhui) Xue. (Corresponding author: Wei Wang.)

Senming Yan and Limin Sun are with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100085, China, and also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing 100085, China (e-mail: yansenming@iie.ac.cn; sunlimin@iie.ac.cn).

Lei Shi is with the School of Integrated Circuits (School of Electronic Information and Electrical Engineering), Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: shi.lei@sjtu.edu.cn).

Wei Wang and Ying Li are with the Department of Strategic and Advanced Interdisciplinary Research, Peng Cheng Laboratory, Shenzhen 518055, China (e-mail: wei\_wang@ieee.org; liying8800@163.com).

Jing Ren is with the Department of Strategic and Advanced Interdisciplinary Research, Peng Cheng Laboratory, Shenzhen 518055, China, and also with the School of Information and Communication Engineering, University of Electronic Science and Technology of China, Chengdu 610054, China (e-mail: renjing@uestc.edu.cn).

Digital Object Identifier 10.1109/TIFS.2026.3666459

## I. INTRODUCTION

MODERN information technologies, such as cloud computing, the Internet of Things (IoT), and generative artificial intelligence (GenAI), are being widely adopted but also introduce new cybersecurity threats. These technologies have become targets of emerging attacks, including ransomware, advanced persistent threats (APTs), and supply chain attacks [1], [2], [3]. Conventional defense systems typically rely on perimeter-based security architectures to mitigate such threats [4], [5], [6]. However, due to the prevalence of insider threats and unknown attacks, perimeter-based defenses are increasingly ineffective, which has driven growing interest in **Zero Trust Architecture (ZTA)** [7], [8]. Guided by the principle of “Never Trust, Always Verify” [9], ZTA does not implicitly trust any user or entity; instead, it continuously evaluates all access requests.

A core technology for realizing ZTA is **User and Entity Behavior Analytics (UEBA)**, which identifies anomalous behavior patterns of users and entities based on the analysis of their behavior records, thereby strengthening zero trust networks [10], [11]. In UEBA systems, operations performed by users and entities are recorded in audit logs and continuously monitored, reducing the risks posed by both insider and external threats. By automatically analyzing audit logs to detect abnormal behaviors, UEBA minimizes the need for manual inspection. Once potentially malicious users or entities are identified, ZTA can trigger appropriate response actions to mitigate security risks.

Existing UEBA methods have incorporated modern data-driven techniques, including classical Machine Learning (ML) and Deep Learning (DL) models, to enhance the feature representation of audit logs [12], [13], [14]. Despite achieving strong detection performance, these methods still face several significant challenges. First, as behavioral records become increasingly large and complex, existing approaches struggle to extract discriminative information that effectively distinguishes anomalous behaviors from normal ones. Second, although both contextual and temporal features are critical for modeling behavior patterns, they are not fully exploited, limiting the ability of current methods to capture complex behavioral patterns. Third, many existing methods rely on labeled data for behavior classification, which hampers their ability to detect previously unseen or unknown malicious activities. Finally, the additional runtime overhead introduced

by DL-based approaches is often overlooked, reducing their practicality for deployment in real-world environments.

To address these challenges, we propose **Large Language Model for Behavior Analytics (LLMBA)**, an LLM-based behavior analytics framework designed to enhance zero trust networks through systematic evaluation of behavior records. Unlike traditional methods that rely on expert knowledge to extract features or employ classic serialization modeling techniques, our goal is to model the distribution of normal behaviors under highly heterogeneous, evolving, and open-world conditions, where attacks manifest as semantic deviations rather than sequential or statistical anomalies. From this perspective, to effectively capture implicit behavioral information, we introduce a multi-level behavior encoding scheme that generates vectorized representations incorporating both temporal and contextual features. Building on these representations, we fine-tune a Llama-2-based behavior learning model to exploit the rich semantic information embedded in the encoded vectors. The LLM serves as a pretrained semantic representation backbone that captures rich contextual and temporal dependencies across events. Owing to the strong representation and generalization capabilities of LLMs, the proposed model can effectively model normal behavioral patterns and accurately detect anomalies. Furthermore, to improve computational and storage efficiency, we employ a knowledge distillation strategy to transfer learned knowledge from the LLM-based model to a lightweight student model. We conduct extensive experiments on the CERT Insider Threat dataset, and the results demonstrate that the proposed LLMB framework consistently outperforms existing methods in detection performance, highlighting the effectiveness and practicality of the LLM-enhanced approach.

To summarize, we have made the following contributions in this work:

- We propose **LLMBA**, an LLM-enhanced behavior analytics framework that fine-tunes the LLaMA-2 foundation model for accurate, efficient behavior learning and anomaly detection. To the best of our knowledge, LLMB represents the first effort to adapt large language models for behavior analytics in zero trust networks.
- We introduce a novel **Multi-Level Behavior Encoding (MLBE)** scheme for LLMB that preserves both contextual and temporal information from behavior records, enabling a more comprehensive understanding of behavioral patterns.
- By leveraging self-supervised fine-tuning, LLMB can effectively detect previously unseen malicious behaviors, making it well-suited for real-world environments characterized by unpredictable and evolving threats.
- To mitigate the runtime overhead of the proposed LLM-enhanced framework, we employ a knowledge distillation strategy to compress the LLM-based model while largely preserving its detection performance.

The remainder of this paper is organized as follows. In Section II, we thoroughly present and compare the existing research works. In Section III, we introduce the key components proposed in the LLMB framework in detail, including the multi-level behavior encoding method, the LLM-

| Activity ID              | Timestamp           | Username | Host    | Activity   |
|--------------------------|---------------------|----------|---------|------------|
| {B9G3-H7BW20EE-4408BFCL} | 07/22/2010 21:48:43 | EHB0824  | PC-0141 | Logon      |
| {K7Z6-E1AC31MP-9153MWLH} | 07/22/2010 22:23:34 | EHB0824  | PC-0141 | Connect    |
| {W2X8-T9W195SF-8448AQIW} | 07/22/2010 22:26:32 | EHB0824  | PC-0141 | HTTP       |
| {X3A4-P7TA28AW-5873TATF} | 07/22/2010 22:28:18 | EHB0824  | PC-0141 | Disconnect |
| {Y1M4-V6SP86AJ-8020APZC} | 07/23/2010 02:10:53 | EHB0824  | PC-0141 | Logoff     |
| {R2K2-E7AI92AW-4573CBQU} | 07/27/2010 06:04:20 | EHB0824  | PC-0141 | Logon      |
| {I4X1-R3CI32YX-6261PUCD} | 07/27/2010 06:29:04 | EHB0824  | PC-0141 | Connect    |
| {M8Q9-F6GB72NB-2039AXBC} | 07/27/2010 06:30:42 | EHB0824  | PC-0141 | HTTP       |
| {Q8E2-V7SD33JI-1121FPVP} | 07/27/2010 07:00:35 | EHB0824  | PC-0141 | Disconnect |
| {F5T8-I7ZS00CS-2468KQZU} | 07/27/2010 07:33:39 | EHB0824  | PC-0141 | Logoff     |

Fig. 1. An example of behavior records generated by a red-team user in the CERT r4.2 dataset. The usernames, timestamps, hosts, and activities are recorded in a structured format.

based behavior learning model, the knowledge distillation-based compression method, and our behavior analytics algorithm. We present the experiment settings in Section IV. We propose the main research questions and analyze the evaluation results in Section V. Finally, conclusions and future work directions are presented in Section VI.

## II. RELATED WORK

To implement behavior analytics in ZTA, the behavior history of users and entities needs to be recorded. Fig. 1 shows an example of preprocessed behavior audit logs, containing behavior records generated by a red-team user from the CERT r4.2 dataset. As users generate massive behavior records over time, which are hardly processed by human experts, ML and DL techniques are leveraged for automatically analyzing this structured data.

The core idea of behavior analytics is to distinguish the patterns between normal and abnormal behaviors. To this end, different behavioral features and various advanced techniques are used in previous works. Some of the existing methods use statistical features from behavior records to detect anomalies. Le and Zincir-Heywood [15] propose to extract numerical features from behavior logs and employ an ensemble of ML-based models for insider threat detection. Lv et al. [12] propose a hybrid framework that leverages an ensemble of Isolation Forest and Markov models to learn both numerical and temporal features. Xu et al. [16] propose to extract numerical features of log events and parameters and use the Principal Component Analysis (PCA) algorithm to detect anomalies. These methods heavily rely on feature extraction methods, which are limited by the predefined rules, making them less effective in processing evolving attacks.

Since behavior records are essentially sequential, structured data, they can be processed in a similar way to natural language. Therefore, in addition to statistical features, some recent works use DL-based sequential models to learn contextual information. Meng et al. [13] propose an end-to-end framework that uses the Long Short Term Memory (LSTM) network to model the sequential behavior patterns and classify

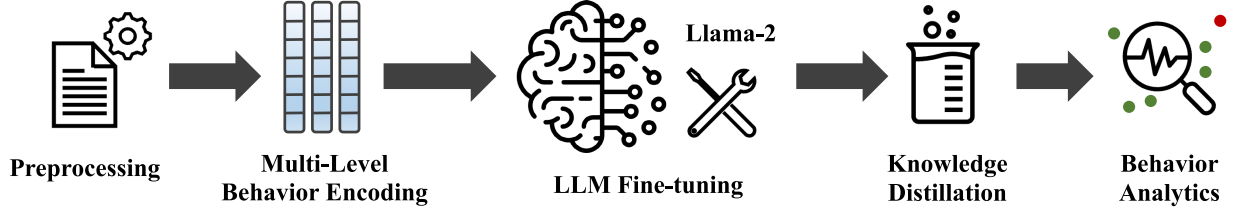


Fig. 2. The pipeline of our proposed LLMB framework.

behavior records. The classification-based approach requires manually labeled categories for training. Different from classification, some researchers design DL-based behavior analytics approaches in a self-supervised manner to reduce the need for labeled data. DeepLog [17] is an LSTM-based method for log anomaly detection, which identifies outlier patterns by making predictions on the following events. Transformer models have shown their advantages in processing sequential data. Huang et al. [14] propose the ITDBERT framework combining BERT-based encoders with LSTM models for threat detection. With the rapid development of Large Language Models (LLMs), some researchers also employ LLMs for analyzing behavior records. Song et al. [18] use the Chain-of-Thought (CoT) reasoning to break down the task and utilize LLM agents to analyze behavior records. Different from these sequential model-based methods, Liu et al. [19] utilize richer contextual information from behavior logs and propose a Log2Vec approach. Log2Vec models user behaviors as heterogeneous graphs, proposes a graph embedding method to acquire compact vectorized data, and uses a clustering algorithm to detect anomalies.

### III. METHODOLOGY

In this section, we introduce the LLMB framework in detail. We present the overview of LLMB, the formal definition of our task, the multi-level behavior encoder, the LLM-based behavior learning model, the knowledge distillation method, and the anomaly detection algorithm for behavior analytics.

#### A. Overview of the LLMB Framework

An overview of the LLMB framework is illustrated in Fig. 2. During the preprocessing stage, LLMB first extracts fixed-length activity sequences from raw behavior records. These sequences are then transformed by a multi-level behavior encoder into vector representations that preserve both behavioral and temporal features. Subsequently, an LLM-based model is designed to learn patterns from the encoded behavior data. The model consists of three main components: an input network that maps the encoded vectors to the LLM input space, multiple pretrained Llama-2 Transformer blocks, and an output network tailored to the behavior learning task. The LLM-based model is trained to predict the next activity given preceding sequences, and the fine-tuned model is ultimately employed for behavior analytics.

#### B. Problem Definition

Behavior analytics aims to determine whether a set of behavior records is normal or abnormal, which can be formulated as a binary classification problem. Formally, we denote a set of original behavior records as  $S$ , where

$$S = (a_1, a_2, \dots, a_l), \quad (1)$$

where  $l$  is the length of the behavior records and  $a_i$  denotes the  $i$ -th activity. Therefore, the goal of the behavior analytics task is to determine whether  $S$  is normal or not.

To fit the LLM-based models, the original sequence is further preprocessed to extract fixed-length activity sequences with length ( $w$ ) as inputs and the next activities as labels. Supposing that  $p$  pairs of inputs and labels are extracted from the original sequence, the preprocessed data is denoted as

$$\begin{aligned} S^{input} &= (s_1^{input}, s_2^{input}, \dots, s_p^{input}), \\ S^{label} &= (s_1^{label}, s_2^{label}, \dots, s_p^{label}), \end{aligned} \quad (2)$$

where

$$\begin{aligned} s_i^{input} &= (a_i^{input}, a_{i+1}^{input}, \dots, a_{i+w-1}^{input}), \\ s_i^{label} &= (a_{i+w}^{input}). \end{aligned} \quad (3)$$

After that, the extracted sequences are vectorized by the Behavior Encoder (BE):

$$X^{input} = \mathbf{BE}(S^{input}). \quad (4)$$

After encoding, we fine-tune an LLM-enhanced model to learn the behavior patterns of normal users, which is denoted as

$$Y^{pred} = \mathbf{BLM}(X^{input}), \quad (5)$$

where  $\mathbf{BLM}$  is the Behavior Learning Model. The fine-tuning procedure is formulated as

$$f_{ft}(S^{label} | Y^{pred}), \quad (6)$$

where  $f_{ft}$  is the fine-tuning algorithm.

After fine-tuning, we use the knowledge distillation technique to compress the BLM, i.e., the teacher model, while retaining its detection performance. Knowledge distillation follows the teacher-student architecture. Supposing that  $Y^{student}$  is the output of the compressed student model and  $f_{kd}$  is the knowledge distillation algorithm, the procedure is denoted as

$$f_{kd}(Y^{pred} | Y^{student}). \quad (7)$$

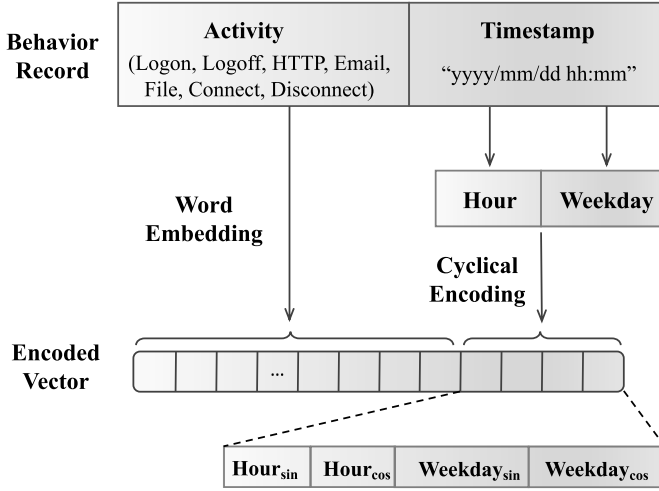


Fig. 3. Illustration of multi-level behavior encoding. The activities are encoded using word embedding, while the temporal features are encoded using cyclical encoding. The encoded features are concatenated as the encoded vector.

### C. Multi-Level Behavior Encoding

The behavior encoder in our framework is designed to transform behavior records into serialized encoded vectors suitable for the LLM-based behavior learning model, as formulated in Equation 4. During the encoding process, we aim to preserve as much information as possible from the original audit data. To this end, we propose a multi-level encoding strategy that jointly captures sequential and temporal features within the LLMB framework. An overview of the proposed multi-level encoding method is illustrated in Fig. 3.

To evaluate user behavior within an organization, it is essential to capture behavior patterns that are sensitive to both activity types and their temporal context. With respect to activity patterns, contextual relationships within activity sequences provide valuable cues for behavior analytics. For instance, a user who frequently accesses removable devices to copy and store files may pose a risk of confidential data exfiltration. Meanwhile, from the temporal perspective, the timing of activities can also signal anomalous behavior. For example, a user who repeatedly logs into internal systems during off-hours may exhibit potentially malicious intent.

In the original behavior records, structured fields such as activity ID, activity name, timestamp, username, host, and other auxiliary information are logged. In the proposed multi-level behavior encoding scheme, we primarily leverage the activity name and timestamp fields. To preserve temporal characteristics, we extract the hour of the day and the day of the week from the timestamp of each behavior record:

$$\mathbf{a} = (\mathbf{a}^{name}, \mathbf{a}^{hour}, \mathbf{a}^{week}), \quad (8)$$

where  $\mathbf{a}^n$ ,  $\mathbf{a}^h$ , and  $\mathbf{a}^w$  denote the **name**, **hour**, and **weekday** of a certain activity, respectively.

Subsequently, we vectorize the extracted features to make them compatible with the LLM-based model. In the LLMB framework, we adopt a multi-level behavior encoding strategy to jointly capture both contextual and temporal information.

1) **Activity-Level Encoding**: For activity names, we employ a word embedding approach to transform the original activity labels into vector representations, enabling the LLMB framework to capture their contextual semantics. We denote this vectorization process as:

$$\mathbf{x}^{name} = \text{Emb}(\mathbf{a}^{name}). \quad (9)$$

2) **Time-Level Encoding**: Unlike activity names, the temporal features—specifically hours and weekdays—are vectorized using a cyclical encoding scheme to capture the inherent periodicity in behavior records. We adopt a Sine-Cosine encoder [20] to separately encode hours and weekdays. For the hour field, which takes integer values from 0 to 23, the resulting encoded vectors are denoted as:

$$\begin{aligned} \mathbf{x}^{h-sin} &= \sin\left(2 * \pi * \frac{\mathbf{a}^{hour}}{24}\right), \\ \mathbf{x}^{h-cos} &= \cos\left(2 * \pi * \frac{\mathbf{a}^{hour}}{24}\right). \end{aligned} \quad (10)$$

For the weekday field, whose value range is an integer between 0 and 6, the encoded vectors are denoted as

$$\begin{aligned} \mathbf{x}^{w-sin} &= \sin\left(2 * \pi * \frac{\mathbf{a}^{week}}{7}\right), \\ \mathbf{x}^{w-cos} &= \cos\left(2 * \pi * \frac{\mathbf{a}^{week}}{7}\right). \end{aligned} \quad (11)$$

After behavioral and temporal encoding, the encoded vectors are concatenated as a whole vector, which is denoted as

$$\mathbf{x}^{input} = (\mathbf{x}^{name}, \mathbf{x}^{h-sin}, \mathbf{x}^{h-cos}, \mathbf{x}^{w-sin}, \mathbf{x}^{w-cos}). \quad (12)$$

Finally, the multi-level behavior encoder generates embedding vectors that capture both behavioral and temporal features, which are subsequently utilized by the LLM-based model for learning.

### D. Adapting LLMs for Behavior Modeling

After multi-level behavior encoding, the encoded vectors contain both contextual and temporal information from the original behavior records, effectively representing serialized data. Using these vectors, we design an LLM-based behavior learning model to capture sequential patterns and detect anomalous behaviors, as formulated in Equation 5.

Since its introduction in 2017, the Transformer architecture has demonstrated significant advantages across a wide range of domains [20], [21], [22]. More recently, the advent of Generative Pre-trained Transformers (GPT) [23], [24] has spurred the development of numerous large language models (LLMs), including Llama [25], Gemini [26], and DeepSeek [27], which have driven substantial advances across multiple fields. Due to their strong ability to model serialized data, LLMs have been increasingly applied to tasks such as time-series analysis [28], mathematical reasoning [29], and wireless communications [30]. Building on these capabilities, we develop an LLM-based behavior learning model within the LLMB framework to effectively capture patterns from the encoded behavior vectors.

The architecture of the behavior learning model in LLMB is illustrated in Fig. 4. We adopt the Meta Llama-2-7B



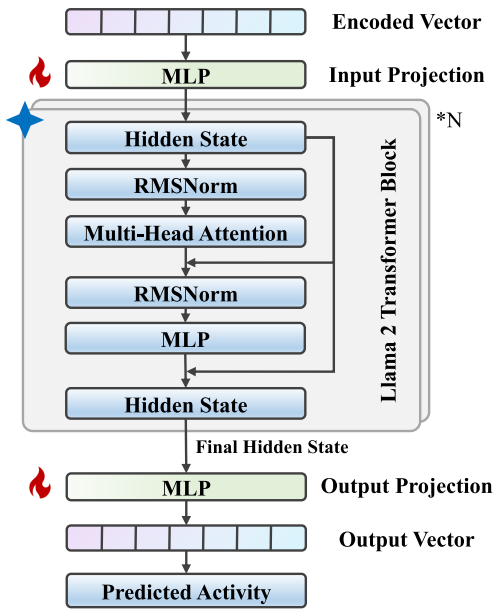


Fig. 4. Illustration of our LLM-based behavior learning model (where the ice symbol represents the frozen components, and the fire icons represent trainable parameters).

foundation model as the backbone and modify its architecture for our task. The model is fine-tuned on an unsupervised prediction task, enabling it to learn normal behavior patterns and detect anomalies. Specifically, the LLM-based behavior learning model is trained to continuously predict the next activity given preceding sequences extracted from normal behavior data. The detailed fine-tuning procedure is described in the following.

As introduced previously, the multi-level behavior encoder transforms raw behavior records into vectors. Let  $X^{input}$  and  $Y^{label}$  denote a batch of vectorized inputs and their corresponding labels for training the BLM, with a batch size of  $b$ . To adapt the encoded vectors to the LLM-based model, we project them to match the LLM input dimensionality. Denoting the dimensions of the encoded vectors and the LLM input layer as  $d^{input}$  and  $d^{LLM}$ , respectively, this projection is performed via fully connected layers, represented as:

$$H_0 = \text{LINEAR}(X^{input}), \quad (13)$$

where

$$X^{input} \in \mathbb{R}^{b \times d^{input}}, H_0 \in \mathbb{R}^{b \times d^{LLM}}. \quad (14)$$

Next, we employ the Llama-2-7B base model as the backbone of our behavior learning model to effectively capture contextual features in behavior records. The Llama-2 model consists of multiple Transformer blocks that process the encoded vectors generated by the multi-level behavior encoder. Compared with the conventional Transformer architecture, Llama introduces several modifications. As described in [25], instead of the standard positional encoding used in Transformers, Llama-2 employs Rotary Position Embedding (RoPE) to improve generalization. In addition, Llama uses Root Mean Square Layer Normalization (RMSNorm) [31] to enhance computational efficiency and stability. We denote

$\text{Transformer\_Block}_n$  as the  $n$ -th Transformer block in our LLM and denote  $H_n$  as its output. The computation of each Transformer block is formulated as

$$H_n = \text{Transformer\_Block}_n(H_{n-1}). \quad (15)$$

To capture the contextual information extracted by the LLM-based backbone, we use the final hidden state of the Llama model. Let the LLM consist of  $N$  Transformer blocks; the final hidden state is denoted as  $H_N$ . To leverage these contextual features for behavior prediction, the LLM-based behavior learning model is trained to predict the next activity in the sequence. Specifically, the output of the LLM backbone is passed through fully connected layers, followed by a Softmax layer to generate the probability distribution over all possible next activities. Assuming there are  $d^{output}$  distinct activities in the behavior records, the output layer of the behavior learning model is defined as

$$Y^{output} = \text{LINEAR}(H_N), \quad (16)$$

where

$$H_N \in \mathbb{R}^{b \times d^{LLM}}, Y^{output} \in \mathbb{R}^{b \times d^{output}}. \quad (17)$$

The behavior learning model is trained to accurately predict the next activity based on the preceding sequence, effectively minimizing the likelihood of incorrect predictions. As the activity prediction task is inherently a multiclass classification problem, we adopt the Cross-Entropy (CE) loss. The loss function used to fine-tune the BLM is formulated as

$$\begin{aligned} Loss_{BLM} &= \text{CE\_Loss}(Y^{output}, Y^{label}) \\ &= - \sum_{i=1}^{d^{output}} Y_i^{label} \log Y_i^{output}. \end{aligned} \quad (18)$$

During training, the parameters of the Llama-2 backbone are frozen, while the appended fully connected layers remain trainable. After fine-tuning, the LLM-based model is deployed for behavior analytics within the LLMBA framework.

#### E. Knowledge Distillation for Model Compression

The high runtime overhead of existing behavior analytics approaches poses a significant barrier to real-world deployment. While our LLM-based behavior learning model demonstrates strong capabilities in modeling sequential data, it also introduces additional computational overhead. To mitigate this issue, we employ **knowledge distillation**, an effective model compression technique that reduces model complexity while preserving detection performance [32], [33].

Knowledge distillation, a form of transfer learning, uses a teacher-student architecture to transfer dark knowledge—the implicit information captured from training data—from a large-scale teacher model to a lightweight student model [34], [35]. During this process, the dark knowledge is typically represented as soft labels, which enhances the generalization capability of the student model [36]. In recent years, numerous variants of knowledge distillation have been developed, including adversarial distillation, multi-teacher distillation, and cross-modal distillation [37], [38], [39].

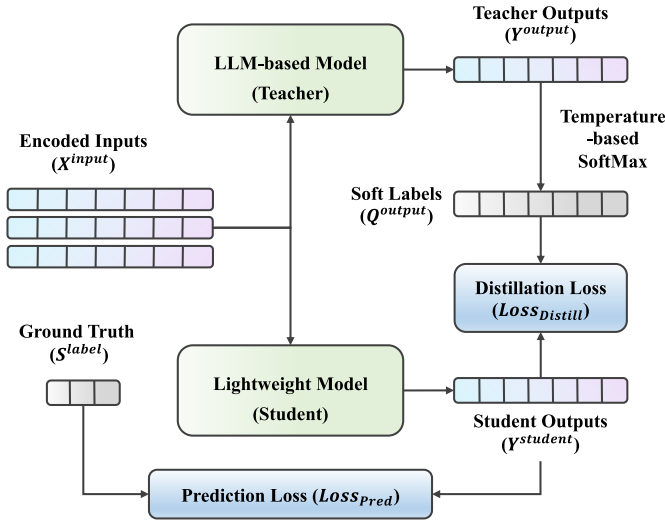


Fig. 5. Illustration of our knowledge distillation approach for low-cost behavior analytics.

In our framework, the LLM-based behavior learning model functions as the teacher, while a lightweight deep learning-based sequential model serves as the student. Through knowledge distillation, the teacher's strong detection capabilities are transferred to the student, significantly reducing the computational overhead of the LLM-based approach. As illustrated in Fig. 5, the proposed knowledge distillation procedure is as follows. First, we construct a lightweight DL-based sequential model, denoted as  $BLM^{light}$ , to serve as the student. Analogous to the LLM-based behavior learning model, the student model is trained to predict the next activity given preceding activity sequences, which is formulated as

$$Y^{student} = BLM^{light}(X^{input}). \quad (19)$$

Next, we extract the dark knowledge from the teacher model. Following the standard knowledge distillation approach [36], a temperature-scaled Softmax function is applied to generate the soft labels, which are computed as

$$Q_i^{output} = \frac{\exp(Y_i^{output}/T)}{\sum_{j=1}^{d^{output}} \exp(Y_j^{output}/T)}, \quad (20)$$

where  $T$  is the temperature value and  $Q_i^{output}$  is the  $i$ -st element in the soft label ( $Q^{output}$ ).

Subsequently, the dark knowledge is transferred to the student model. To enhance the generalization capability of the lightweight model, the student is trained to match the soft labels generated by the teacher. The Kullback–Leibler (KL) divergence is used to measure the difference between the soft labels and the student model's outputs, and the student is optimized to minimize this distance. The resulting distillation loss is denoted as

$$\begin{aligned} Loss_{Distill} &= D_{KL}(Y^{student} || Q^{output}) \\ &= \sum_i^{d^{output}} Y_i^{student} \log \frac{Y_i^{student}}{Q_i^{output}}. \end{aligned} \quad (21)$$

In addition to the distillation loss, the student model is trained to accurately predict the next activities, ensuring it fulfills the behavior learning task. Analogous to the LLM-based behavior learning model, the prediction loss for training the student model is denoted as

$$\begin{aligned} Loss_{Pred} &= CE\_Loss(Y^{student}, Y^{label}) \\ &= - \sum_{i=1}^{d^{output}} Y_i^{label} \log Y_i^{student}. \end{aligned} \quad (22)$$

Then, the overall loss function of knowledge distillation is formulated as

$$Loss_{KD} = \alpha \cdot Loss_{Distill} + \beta \cdot Loss_{Pred}, \quad (23)$$

where  $\alpha$  and  $\beta$  are the hyperparameters controlling the ratio of two loss functions. The student model is trained to transfer the dark knowledge from the LLM-based teacher model using the loss function above. After knowledge distillation, the distilled student model is used for the behavior analytics task with low runtime overhead.

#### F. Deploying LLMBa for Behavior Analytics

We describe our behavior analytics approach in the LLBA framework in the following. The LLBA framework uses the previously fine-tuned and compressed behavior model for recognizing anomalous patterns and identifying malicious behaviors. As shown in Algorithm 1, we design an online detection algorithm for behavior analytics.

##### Algorithm 1 LLMBa for Behavior Analytics

**Input** :  $S^{origin}$ ,  $K$ ,  $w$ , and  $\tau$ .

**Output**: Behavior analytics result of  $S^{origin}$ .

```

1 Initialize  $R$  to store results of subsequences;
2 for  $i \leftarrow 1$  to  $L - w$  do
    // extract input and label
3    $S_i^{input} \leftarrow S^{origin}[i : i + w]$ ;
4    $S_i^{label} \leftarrow S^{origin}[i + w]$ ;
    // behavior encoding and predicting
5    $X_i^{en} \leftarrow BE(S_i^{input})$ ;
6    $Y_i^{pred} \leftarrow BLM(X_i^{en})$ ;
7    $P_i \leftarrow \text{Top-K}(Y_i^{pred}, K)$ ;
8   if  $S_i^{label} \notin P_i$  then
9      $r_i = 1$ ;
10  else
11     $r_i = 0$ ;
12  $R = \text{avg}(r)$ ;
13 if  $R > \tau$  then
14   Anomaly Detected.
```

Specifically, given a sequence of behavior records as input ( $S^{origin}$ ), we iteratively extract  $p$  pairs of fixed-length subsequences ( $S^{input}$ ) and corresponding ground-truth labels ( $S^{label}$ ).

For a single pair of input and label ( $S_i^{input}$  and  $S_i^{label}$ ), we first use our multi-level BE to get the encoded input. Then, we use the LLM-enhanced BLM to output the probability distribution of the next activities ( $Y_i^{pred}$ ). From the probability distribution, we extract  $K$  indexes of activities with the highest probabilities in descending order, which are denoted as

$$P_i = (a_i^1, a_i^2, \dots, a_i^K) = \text{Top-K}(Y_i^{pred}), \quad (24)$$

where  $A_i^k$  is the activity name with  $k$ -highest probability in the  $i$ -th behavior sequence. Since our LLM-based behavior learning model is fine-tuned to learn the sequential features in normal data, we identify abnormal behavior patterns by comparing the predicted Top-K activities with the ground-truth label, where  $K$  is a preset parameter related to the number of activity categories. An anomalous pattern is detected if a ground-truth label is not in the corresponding Top-K activities. We denote normal as 0 and abnormal as 1, then the detection result ( $r_i$ ) of a single subsequence is formulated as

$$r_i = \begin{cases} 1, & \text{if } S_i^{label} \notin P_i; \\ 0, & \text{otherwise.} \end{cases} \quad (25)$$

Then, we define the risk score ( $R$ ) of a behavior sequence as the ratio of its abnormal subsequences:

$$R = \frac{1}{p} \sum_i^p r_i. \quad (26)$$

Finally, LLMB uses a threshold-based detection approach for behavior analytics, where the threshold value is preset based on expert knowledge. An anomalous sequence is detected if its risk score is above the threshold ( $\tau$ ).

#### IV. EVALUATION SETTINGS

##### A. Evaluation Datasets

We conduct the experiments on the Insider Threat Test Dataset (CERT Dataset) released by Carnegie Mellon University [40]. The CERT dataset is a collection of synthetic insider threat test datasets that record the behaviors of both normal and malicious users within an enterprise [41]. Since the CERT r4.2 and r5.2 datasets contain rich red-team data, we evaluate our proposed LLMB framework on them.

In the CERT r4.2 and r5.2 datasets, four different insider attack scenarios are simulated:

- 1) Scenarios #1: inner users begin to log in after hours, use removable drives, and upload data to an online website.
- 2) Scenarios #2: inner users search for jobs from competitors and use thumb drives to steal data.
- 3) Scenarios #3: malicious users take control of the supervisors' accounts and send out emails to cause panic.
- 4) Scenarios #4: users log into other users' devices, search for private files, and email them to their own accounts.

The CERT r4.2 dataset contains behavior records generated by 1,000 users, including 930 normal users and 70 malicious users. The malicious users simulate the first three attack scenarios described above. Over a period of 74 weeks, a total of 32,770,227 operations performed by these users

were recorded. The CERT r5.2 dataset, in contrast, comprises behavior records from 2,000 internal users collected over the same 74-week period. This dataset includes 1,901 benign users and 99 malicious users, with the malicious users simulating all four attack scenarios.

##### B. Dataset Preprocessing

The CERT r4.2 and r5.2 datasets store different types of behavior records in separate files (e.g., logons, network records, file operations, etc.). Therefore, we first extract the activities from these files and combine them by username. Then, all the activities are sorted by timestamp. Additionally, since our proposed behavior encoding method leverages multi-level temporal information, we extract the hours and weekdays from the original timestamps.

In the first experiment, for CERT r4.2, we randomly select 100 users from the normal users for testing and the other 830 users for training. For CERT r5.2, considering the dataset scale, we randomly select 20 benign users for training and 100 for testing. Meanwhile, all the malicious behavior records in these two datasets are tested. Besides, as our model takes fixed-length sequences as inputs, we use a sliding window with  $W = 10$  to iteratively extract activity sequences as inputs and the next activities as labels. In the second experiment, to evaluate the performance of LLMB in detecting changing behavior patterns, we use the normal and abnormal data generated by all malicious users for testing.

##### C. Baselines

In our experiments, we compare the LLMB framework with some advanced baselines. We give a brief introduction to these methods as follows.

1) *PCA*: Principal Component Analysis (PCA) is a widely applied ML method for extracting significant features from input data. PCA is originally used for dimensionality reduction. It can also detect anomalies by reconstructing the original data and measuring the reconstruction loss.

2) *Isolation Forest*: Isolation Forest (IF) [42] is an unsupervised ML algorithm for outlier detection. IF aims to isolate abnormal data points and assign anomaly scores with the outputs of a set of decision trees. IF has a linear time complexity and low runtime overhead.

3) *Anomaly Clustering*: Anomaly clustering algorithms aim to cluster data and identify outlier clusters. In this work, we implement the LogCluster method [43], an anomaly clustering approach in the log anomaly detection domain. It extracts statistical features from behavior records and then clusters these records.

4) *DeepLog*: DeepLog [17] is the first DL-based approach for anomaly detection in the log analysis domain. DeepLog uses multiple LSTM layers to learn the sequential features from serialized data. After training, DeepLog detects abnormal sequential patterns based on its predictions.

5) *ITDBERT*: ITDBERT [14] uses both Word2Vec and BERT to encode the temporal information in behavior records. Then, ITDBERT employs the LSTM-based model to learn sequential features and classify user behaviors. The original

ITDBERT model is trained for binary classification with supervised learning. In this work, we modify the ITDBERT architecture to train a prediction-based anomaly detector, similar to the DeepLog method.

6) *Transformer*: We implement a Transformer-based behavior learning model, combining our proposed behavior encoding approach and a multi-layer Transformer encoder-based model. Similar to DeepLog, it also leverages a prediction-based anomaly detection method.

7) *LogBERT*: LogBERT [44] employs BERT [45] to automatically analyze system log data. Analogous to the masked language modeling paradigm, LogBERT detects anomalies by evaluating the prediction likelihoods of masked log events.

8) *Qwen*: We implement a Qwen-based behavior learning model, which shares a similar architecture with LLMBA with a lighter base model, i.e., Qwen-3-1.7B.<sup>1</sup>

#### D. Implementation Details

In this work, we use the Meta Llama-2-7B<sup>2</sup> model as the backbone of our behavior learning model. In the preprocessing stage, we use a sliding window with  $W = 10$  to acquire fixed-length input sequences. After behavior encoding, the user activities are represented as 20-dimensional vectors, comprising 16 dimensions for activity type, 2 for hour information, and 2 for weekday information ( $d^{input} = 20$ ). These input vectors are subsequently projected to match the hidden size of Llama ( $d^{LLM} = 4096$ ). Finally, the dimensionality of the Llama outputs is reduced to the number of activity types, yielding an output dimension of  $d^{output} = 7$ .

For hyperparameter settings, limited by computing resources, we train LLMBA and Qwen-based model with  $batch\_size = 128$ ,  $epoch = 10$ , and  $learning\_rate = 3e^{-4}$  to acquire the best performance. We use AdamW to optimize these LLM-based models. Besides, we set  $K = 1$  for our behavior analysis algorithm. For other baselines, we implement the DeepLog and ITDBERT models with  $num\_layers = 2$  and  $hidden\_size = 128$ . For the Transformer model, we set  $num\_heads = 2$ ,  $num\_layers = 2$ , and  $hidden\_size = 128$ . For LogBERT, we set  $num\_heads = 4$ ,  $num\_layers = 4$ , and  $hidden\_size = 128$ . We train all these DL-based baselines for 100 epochs with  $batch\_size = 512$  and  $learning\_rate = 5e^{-3}$  to achieve the best performance.

For knowledge distillation, we set  $Temp = 0.1$ ,  $\alpha = 1.0$ , and  $\beta = 1.0$  in the loss function. We train some DL-based sequential models as the student in the distillation process. We train these student models for 10 epochs with  $batch\_size = 512$  and  $learning\_rate = 5e^{-3}$  to achieve the best performance.

We conduct the experiments on a server with Ubuntu 20.04, an Intel Xeon CPU, 128GB memory, and NVIDIA A6000 GPUs. We implement all the methods with CUDA 11.8, Python 3.9, and PyTorch 2.5.

#### E. Evaluation Metrics

We count four indicators, including True Positives (TP), False Positives (FP), True Negatives (TN), and False Negatives

(FN), and then calculate three metrics to evaluate all the methods, including  $Precision = \frac{TP}{TP+FP}$ ,  $Recall = \frac{TP}{TP+FN}$ , and  $F1\ Score = 2 * \frac{Precision * Recall}{Precision + Recall}$ .

We also compute the area under the receiver operating characteristic curve (AUC) to assess the ability of different detectors to discriminate between normal and malicious data. In addition, to evaluate the effectiveness of the model compression method, we consider three metrics to quantify the runtime overhead of log-based anomaly detectors: the number of parameters (#Params), the number of floating-point operations (FLOPs), and the inference time per input sequence.

### V. EVALUATION AND ANALYSIS

In this section, we aim to answer the following research questions to evaluate the performance of our proposed method:

**RQ1.** How is the overall performance of our proposed LLMBA framework compared to the baseline methods?

**RQ2.** Is the LLMBA framework able to detect unknown malicious behavior patterns?

**RQ3.** Is the LLMBA framework able to identify the changes in user behavior patterns?

**RQ4.** How effective is our knowledge distillation approach in compressing the behavior learning model?

#### A. Evaluation of User-Level Anomaly Detection (RQ1 & RQ2)

In the first experiment, we evaluate the overall detection performance of the proposed LLMBA framework on the CERT r4.2 and r5.2 datasets. All evaluated methods are trained exclusively on benign data and tested on a mixture of benign and malicious data. The objective of all methods is to distinguish benign user behaviors from malicious ones. We compare the LLMBA framework with several baseline approaches using the evaluation metrics described in Section IV.

The evaluation results are presented in Table I. Overall, the proposed LLMBA framework outperforms all evaluated methods across both datasets, achieving an F1 score of 99.41% and an AUC of 1.000 on CERT r4.2, as well as an F1 score of 92.31% and an AUC of 0.961 on CERT r5.2. These results demonstrate the effectiveness of our approach in learning discriminative behavior patterns that reliably distinguish normal users from malicious ones.

For the classical ML-based baselines, including PCA, Isolation Forest, Anomaly Clustering, and One-Class SVM, the experimental results indicate consistently low detection performance. In particular, the One-Class SVM model is almost unable to distinguish malicious users on both datasets. These findings suggest that, although classical ML models are effective at learning from numerical features, they struggle to capture contextual information, which is crucial for behavior analytics tasks.

The DL-based methods, including DeepLog, ITDBERT, Transformer, LogBERT, Qwen, and LLMBA, achieve substantially higher performance than the ML-based baselines, indicating that deep sequential models are more effective for modeling serialized behavior records. The Transformer model implemented in our experiments adopts an architecture

<sup>1</sup><https://huggingface.co/Qwen/Qwen3-1.7B>

<sup>2</sup><https://huggingface.co/meta-llama/Llama-2-7b>



TABLE I  
EVALUATION OF OVERALL PERFORMANCE ON CERT R4.2 AND R5.2 DATASETS

| Dataset<br>Method  | CERT r4.2 |         |          |           | CERT r5.2 |         |          |           |
|--------------------|-----------|---------|----------|-----------|-----------|---------|----------|-----------|
|                    | Precision | Recall  | F1 Score | AUC Score | Precision | Recall  | F1 Score | AUC Score |
| PCA                | 29.58%    | 100.00% | 45.65%   | 0.500     | 47.09%    | 100.00% | 64.03%   | 0.500     |
| Isolation Forest   | 14.28%    | 7.14%   | 9.52%    | 0.446     | 55.68%    | 55.06%  | 55.37%   | 0.580     |
| Anomaly Clustering | 100.00%   | 7.14%   | 13.33%   | 0.536     | 98.39%    | 68.54%  | 80.79%   | 0.838     |
| One-Class SVM      | 0.00%     | 0.00%   | NaN      | 0.300     | 5.66%     | 6.74%   | 6.15%    | 0.239     |
| DeepLog            | 97.56%    | 57.14%  | 72.07%   | 0.826     | 89.11%    | 90.91%  | 90.00%   | 0.948     |
| ITDBERT            | 80.00%    | 97.14%  | 87.74%   | 0.901     | 87.50%    | 91.92%  | 89.66%   | 0.895     |
| Transformer        | 88.46%    | 98.57%  | 93.24%   | 0.981     | 89.11%    | 90.91%  | 90.00%   | 0.945     |
| LogBERT            | 95.83%    | 98.57%  | 97.18%   | 0.994     | 89.11%    | 90.91%  | 90.00%   | 0.945     |
| Qwen               | 97.22%    | 100.00% | 98.59%   | 1.000     | 89.00%    | 89.90%  | 89.45%   | 0.940     |
| LLMBA (ours)       | 98.59%    | 100.00% | 99.29%   | 1.000     | 93.75%    | 90.91%  | 92.31%   | 0.961     |

similar to DeepLog but replaces recurrent components with Transformer encoder layers to better capture contextual dependencies. As a result, the Transformer consistently outperforms DeepLog and ITDBERT in terms of both F1 score and AUC, demonstrating the superior representation capacity of the Transformer architecture.

Notably, the proposed LLMB framework achieves the highest performance across all evaluation metrics among the DL-based approaches. On the CERT r4.2 dataset, LLMB improves the F1 score by 2.17% and the AUC by 0.6% compared with LogBERT, a state-of-the-art log anomaly detection method. Furthermore, while LLMB achieves performance comparable to Qwen on CERT r4.2, it delivers a significant improvement on CERT r5.2, highlighting the strong capability of the deployed Llama-2 model in representing complex contextual features. Overall, these results demonstrate that the LLMB framework effectively distinguishes normal from anomalous behavior patterns and achieves high detection accuracy with fewer false alarms, making it well-suited for real-world deployment.

More importantly, the LLM-based behavior learning model is trained exclusively on benign data; therefore, the behavior patterns of malicious users in the test set are entirely unseen by the LLMB framework. The experimental results shown in Table I demonstrate the superior capability of the proposed framework in identifying previously unseen malicious behavior patterns, making LLMB effective in handling unpredictable and evolving attack strategies.

In summary, the first experiment shows that the proposed LLMB framework achieves high detection performance while effectively capturing the behavioral characteristics of malicious users. Moreover, it confirms that LLMB can robustly detect unknown malicious behaviors, highlighting its practical applicability in real-world security scenarios.

#### B. Detection of Changes in Behavior Patterns (RQ3)

In ZTA, trust is never implicitly granted; consequently, zero trust networks do not distinguish between internal and external users. All user behaviors are continuously recorded

and monitored, as insider users may also engage in malicious activities. In this context, the second experiment evaluates the capability of the proposed LLMB framework to detect changes in user behavior over time.

In the CERT r4.2 and r5.2 datasets, researchers simulate scenarios in which internal employees transition from benign users to malicious attackers. The timestamps at which users begin to exhibit abnormal behaviors are explicitly labeled, enabling the partitioning of red-team users' activity sequences into normal and abnormal segments. This setup allows us to assess the effectiveness of all evaluated methods in identifying evolving behavioral patterns. In this experiment, both benign and malicious records from red-team users are extracted as labeled test data, and the evaluated methods aim to distinguish between these two classes of behaviors.

The evaluation results are presented in Table II. Consistent with the first experiment, the ML-based baselines exhibit relatively low detection performance, while the DL-based methods generally outperform their ML counterparts. Overall, the proposed LLMB framework achieves the highest detection performance across both datasets. Notably, LLMB attains performance comparable to Qwen on the CERT r4.2 dataset, while significantly outperforming it on CERT r5.2. In addition, on CERT r5.2, the Transformer and LogBERT models achieve marginally higher AUC scores than our proposed LLMB framework (by approximately 0.1%); however, LLMB consistently outperforms these methods across the remaining evaluation metrics on both datasets.

In summary, the second experiment demonstrates that the proposed LLMB framework can effectively detect evolving behavior patterns of red-team users. Owing to its strong detection performance, LLMB is well-suited to enhance zero trust networks through continuous monitoring and analysis of user behavior.

#### C. Evaluation of Model Compression (RQ4)

As ZTA requires continuous monitoring and analysis of user behavior, the runtime efficiency of behavior analytics approaches is critical for real-world deployment. In LLMB,

TABLE II  
DETECTION OF CHANGES IN BEHAVIOR PATTERNS ON CERT R4.2 AND R5.2 DATASETS

| Dataset<br>Method  | CERT r4.2 |         |               |              | CERT r5.2 |         |               |              |
|--------------------|-----------|---------|---------------|--------------|-----------|---------|---------------|--------------|
|                    | Precision | Recall  | F1 Score      | AUC Score    | Precision | Recall  | F1 Score      | AUC Score    |
| PCA                | 37.50%    | 100.00% | 54.55%        | 0.500        | 47.34%    | 100.00% | 64.26%        | 0.500        |
| Isolation Forest   | 7.69%     | 7.14%   | 7.41%         | 0.279        | 45.79%    | 55.06%  | 50.00%        | 0.482        |
| Anomaly Clustering | 100.00%   | 7.14%   | 13.33%        | 0.536        | 100.00%   | 68.54%  | 81.33%        | 0.843        |
| One-Class SVM      | 0.00%     | 0.00%   | NaN           | 0.179        | 5.71%     | 6.74%   | 6.19%         | 0.140        |
| DeepLog            | 97.56%    | 57.14%  | 72.07%        | 0.748        | 87.00%    | 87.88%  | 87.44%        | 0.907        |
| ITDBERT            | 87.30%    | 78.57%  | 82.71%        | 0.836        | 88.35%    | 91.92%  | 90.10%        | 0.899        |
| Transformer        | 93.44%    | 81.43%  | 87.02%        | 0.980        | 87.13%    | 88.89%  | 88.00%        | <b>0.940</b> |
| LogBERT            | 89.33%    | 95.71%  | 92.41%        | 0.967        | 87.13%    | 88.89%  | 88.00%        | <b>0.940</b> |
| Qwen               | 98.53%    | 95.71%  | 97.10%        | <b>0.997</b> | 85.57%    | 83.84%  | 84.69%        | 0.890        |
| LLMBA (ours)       | 98.55%    | 97.14%  | <b>97.84%</b> | <b>0.997</b> | 88.24%    | 90.91%  | <b>89.55%</b> | 0.939        |

TABLE III  
EVALUATION OF RUNTIME EFFICIENCY ON THE CERT R4.2 DATASET

| Model                    | F1 Score | AUC Score | FLOPs   | #Params | InferTime |
|--------------------------|----------|-----------|---------|---------|-----------|
| DeepLog                  | 77.06%   | 0.826     | 2.09M   | 207.75K | 3.64ms    |
| ITDBERT                  | 88.82%   | 0.901     | 0.62M   | 70.82K  | 3.64ms    |
| Transformer              | 94.12%   | 0.981     | 85.62K  | 9.74K   | 1.03ms    |
| LogBERT                  | 97.18%   | 0.994     | 0.17M   | 17.37K  | 1.90ms    |
| Qwen                     | 98.59%   | 1.000     | 17.20B  | 1.72B   | 33.20ms   |
| Teacher (MLBE+Llama)     | 99.29%   | 1.000     | 66.07B  | 6.61B   | 47.76ms   |
| Student #1 (Transformer) | 97.14%   | 0.994     | 107.02K | 12.10K  | 1.03ms    |
| Student #2 (Transformer) | 92.41%   | 0.967     | 15.82K  | 2.82K   | 0.11ms    |
| Student #3 (LSTM)        | 97.18%   | 0.994     | 2.11M   | 209.80K | 4.11ms    |
| Student #4 (LSTM)        | 71.43%   | 0.876     | 71.92K  | 7.14K   | 3.82ms    |

we adopt a knowledge distillation approach to compress the proposed LLM-enhanced behavior learning model. To evaluate the effectiveness of this distillation strategy, we conduct a series of experiments using the dataset from the first experiment (i.e., the overall performance evaluation). We implement several Transformer- and LSTM-based models as student networks within the distillation framework. The detailed architectures of these student models are described as follows:

- Student Model #1: a multi-layer Transformer Encoder-based lightweight BLM with  $num\_layer = 2$ ,  $num\_head = 2$ , and  $hidden\_size = 128$ .
- Student Model #2: a Transformer Encoder-based lightweight BLM with  $num\_layer = 1$ ,  $num\_head = 1$ , and  $hidden\_size = 32$ .
- Student Model #3: a multi-layer LSTM-based lightweight BLM with  $num\_layer = 2$  and  $hidden\_size = 128$ .
- Student Model #4: an LSTM-based lightweight BLM with  $num\_layer = 1$  and  $hidden\_size = 32$ .

Following the procedure described in Section III, we transfer knowledge from the LLM-enhanced teacher model to the student models. Both performance and efficiency metrics are

evaluated for the teacher and student models to assess the effectiveness of the proposed framework. The results of the knowledge distillation experiments are presented in Table III.

Student #1 adopts the same architecture as the Transformer-based baseline evaluated in the first experiment. After knowledge distillation, Student #1 significantly outperforms the Transformer baseline, achieving 97.65% accuracy, a 99.29% F1 score, and an AUC of 0.994. This improvement demonstrates that the dark knowledge embedded in the LLM-enhanced teacher model is effectively transferred to the student through the proposed distillation approach. Similarly, the LSTM-based student model (Student #3) exhibits a substantial performance gain after distillation, reaching 97.65% accuracy, a 97.18% F1 score, and an AUC of 0.994.

Importantly, although these DL-based student models achieve slightly lower detection performance than the LLM-based teacher, they incur significantly reduced computational and storage overhead. In particular, the Transformer-based student (Student #1) preserves 97.83% of the teacher model's detection performance while reducing FLOPs, parameter count, and inference time by over 99%. These results confirm

TABLE IV  
ABLATION STUDY ON ENCODING APPROACH

| Dataset      | Method               | F1 Score      | AUC Score    |
|--------------|----------------------|---------------|--------------|
| CERT<br>r4.2 | MLBE (Onehot)+Llama  | 95.03%        | 0.988        |
|              | Word Embedding+Llama | 97.14%        | 0.993        |
|              | MLBE+Llama (ours)    | <b>99.29%</b> | <b>1.000</b> |
| CERT<br>r5.2 | MLBE (Onehot)+Llama  | 91.54%        | 0.958        |
|              | Word Embedding+Llama | 87.00%        | 0.929        |
|              | MLBE+Llama (ours)    | <b>92.31%</b> | <b>0.961</b> |

that knowledge distillation effectively mitigates the runtime overhead associated with the LLM-based behavior learning model.

Furthermore, our results indicate that, while dark knowledge can be successfully transferred, the performance of student models remains constrained by their model capacity. As shown in Table III, Student #2 and Student #4, which have more compact architectures, achieve lower performance after distillation compared with larger student models.

In summary, this experiment demonstrates that the proposed knowledge distillation approach can effectively compress the LLM-based behavior learning model while preserving high detection performance. After distillation, the resulting lightweight behavior learning models are significantly more suitable for practical deployment in real-world environments with constrained computational and storage resources.

#### D. Ablation Study on Encoding Approach

To evaluate the effectiveness of the proposed multi-level behavior encoding approach, we replace the encoding module in LLMB with the following alternatives: (I) a combination of one-hot encoding and cyclic encoding, and (II) word embeddings for activity names. The modified behavior learning models are retrained and evaluated under the same experimental settings described in Section IV.

The experimental results are presented in Table IV. The results show that the proposed multi-level behavior encoding approach, which integrates word embeddings with cyclic encoding, provides richer contextual information to the LLM and leads to superior detection performance on both datasets. These findings confirm the effectiveness of the proposed behavior learning approach.

#### E. Ablation Study on Model Architecture

In our proposed LLMB framework, Llama-2-7B is employed as the backbone of the behavior learning model. To evaluate the effectiveness of this design, we replace the Llama backbone with alternative models, including a multi-layer LSTM, a multi-layer Transformer, and a Qwen LLM, while keeping the encoding methods and all other experimental settings identical to the original LLMB configuration.

The evaluation results are presented in Table V. Empirical results on both datasets indicate that, when enhanced with

TABLE V  
ABLATION STUDY ON MODEL ARCHITECTURE

| Dataset      | Method            | F1 Score      | AUC Score    |
|--------------|-------------------|---------------|--------------|
| CERT<br>r4.2 | MLBE+LSTM         | 97.14%        | 0.994        |
|              | MLBE+Transformer  | 93.24%        | 0.981        |
|              | MLBE+Qwen         | 98.59%        | <b>1.000</b> |
|              | MLBE+Llama (ours) | <b>99.29%</b> | <b>1.000</b> |
| CERT<br>r5.2 | MLBE+LSTM         | 85.26%        | 0.917        |
|              | MLBE+Transformer  | 90.00%        | 0.945        |
|              | MLBE+Qwen         | 89.45%        | 0.940        |
|              | MLBE+Llama (ours) | <b>92.31%</b> | <b>0.961</b> |

Llama, the proposed LLMB framework significantly outperforms the baseline models, highlighting the strong capability of LLMs in modeling sequential features. These findings demonstrate that the Llama-2 language model adopted in LLMB is effective at capturing implicit and complex patterns in user activities.

#### F. Limitations

Although the proposed LLMB framework achieves strong performance in behavior analytics, several limitations remain. First, the LLM-based architecture introduces additional overhead during fine-tuning of the Llama model, resulting in higher storage and computational costs compared with traditional ML- and DL-based methods. Nevertheless, we argue that this overhead is justified by the substantial improvement in detection performance. Second, owing to its end-to-end LLM-based design, the framework offers limited interpretability relative to classical machine learning approaches, which may affect the transparency and trustworthiness of its analytical outcomes. Finally, like other DL-based systems, the proposed framework may be vulnerable to AI-driven attacks, such as adversarial attacks, backdoor attacks, and data poisoning [46]. Adversaries could potentially exploit these weaknesses to evade detection and bypass zero trust defenses. Addressing such adversarial threats is left for future work.

## VI. CONCLUSION AND FUTURE WORK

In this work, we propose **LLMB**, an LLM-enhanced behavior analytics framework for zero trust networks. We introduce a novel multi-level behavior encoding scheme that transforms behavioral data into vector representations while preserving both contextual and temporal information. Furthermore, we fine-tune an LLM-based behavior learning model to leverage the strong representation and generalization capabilities of large language models. The model is trained in a self-supervised manner, enabling the detection of previously unseen behaviors. To reduce the computational and storage overhead of the LLM-based model, we further employ a knowledge distillation strategy that compresses the model while largely retaining its detection performance. In addition, we design a prediction-based detection algorithm tailored

for behavior analytics. Extensive experiments on the CERT datasets demonstrate the effectiveness and advantages of the proposed LLMBA framework.

For future work, several research directions merit further investigation. First, the adoption of DL models in behavior analytics introduces new security vulnerabilities, such as adversarial attacks and backdoor attacks. Accordingly, future research should explore robust adversarial attack and defense mechanisms for DL-based behavior analytics systems. Second, the limited interpretability of end-to-end deep models remains a key challenge that hinders their practical adoption. Beyond achieving high detection accuracy, future work should focus on developing interpretable behavior analytics methods to improve transparency and trustworthiness.

## REFERENCES

- [1] I. H. Sarker, A. I. Khan, Y. B. Abushark, and F. Alsolami, "Internet of Things (IoT) security intelligence: A comprehensive overview, machine learning solutions and research directions," *Mobile Netw. Appl.*, vol. 28, no. 1, pp. 296–312, Feb. 2023.
- [2] F. K. Parast, C. Sindhav, S. Nikam, H. I. Yekta, K. B. Kent, and S. Hakak, "Cloud computing security: A survey of service-based models," *Comput. Secur.*, vol. 114, Mar. 2022, Art. no. 102580.
- [3] I. Nevat et al., "Anomaly detection and attribution in networks with temporally correlated traffic," *IEEE/ACM Trans. Netw.*, vol. 26, no. 1, pp. 131–144, Feb. 2018.
- [4] R. Rais, C. Morillo, E. Gilman, and D. Barth, *Zero Trust Networks: Building Secure Systems in Untrusted Networks*. Sebastopol, CA, USA: O'Reilly Media, 2024.
- [5] X. Wang, W. Shi, Y. Xiang, and J. Li, "Efficient network security policy enforcement with policy space analysis," *IEEE/ACM Trans. Netw.*, vol. 24, no. 5, pp. 2926–2938, Oct. 2016.
- [6] S. Chen and Q. Song, "Perimeter-based defense against high bandwidth DDoS attacks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 16, no. 6, pp. 526–537, Jun. 2005.
- [7] Y. He, D. Huang, L. Chen, Y. Ni, and X. Ma, "A survey on zero trust architecture: Challenges and future trends," *Wireless Commun. Mobile Comput.*, vol. 2022, no. 1, Jan. 2022, Art. no. 6476274.
- [8] S. Hong, L. Xu, J. Huang, H. Li, H. Hu, and G. Gu, "SysFlow: Toward a programmable zero trust framework for system security," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 2794–2809, 2023.
- [9] V. Stafford, "Zero trust architecture," *NIST Special Publication*, vol. 800, no. 207, pp. 207–800, 2020.
- [10] S. Khaliq, Z. U. Abideen Tariq, and A. Masood, "Role of user and entity behavior analytics in detecting insider attacks," in *Proc. Int. Conf. Cyber Warfare Secur. (ICWS)*, Oct. 2020, pp. 1–6.
- [11] R. Freter, "Zero trust reference architecture," U.S. Dept. Defense, Washington, DC, USA, Tech. Rep., DoD Zero Trust Reference Architecture, Version 2.0, 2022.
- [12] B. Lv, D. Wang, Y. Wang, Q. Lv, and D. Lu, "A hybrid model based on multi-dimensional features for insider threat detection," in *Proc. WASA*, vol. 10874, Tianjin, China, Jun. 2018, pp. 333–344.
- [13] F. Meng, F. Lou, Y. Fu, and Z. Tian, "Deep learning based attribute classification insider threat detection for data security," in *Proc. IEEE 3rd Int. Conf. Data Sci. Cyberspace (DSC)*, Guangzhou, China, Jun. 2018, pp. 576–581.
- [14] W. Huang, H. Zhu, C. Li, Q. Lv, Y. Wang, and H. Yang, "ITDBERT: Temporal-semantic representation for insider threat detection," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Sep. 2021, pp. 1–7.
- [15] D. C. Le and N. Zincir-Heywood, "Anomaly detection for insider threats using unsupervised ensembles," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 2, pp. 1152–1164, Jun. 2021.
- [16] W. Xu, L. Huang, A. Fox, D. Patterson, and M. Jordan, "Online system problem detection by mining patterns of console logs," in *Proc. 9th IEEE Int. Conf. Data Mining*, Miami, FL, USA, Dec. 2009, pp. 588–597.
- [17] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 1285–1298.
- [18] C. Song, L. Ma, J. Zheng, J. Liao, H. Kuang, and L. Yang, "Audit-LLM: Multi-agent collaboration for log-based insider threat detection," 2024, *arXiv:2408.08902*.
- [19] F. Liu, Y. Wen, D. Zhang, X. Jiang, X. Xing, and D. Meng, "Log2vec: A heterogeneous graph embedding based approach for detecting cyber threats within enterprise," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2019, pp. 1777–1794.
- [20] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inform. Process. Syst. (NIPS)*, 2017, pp. 5998–6008.
- [21] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," in *Proc. ICLR*, May 2021, pp. 1–16.
- [22] F. Sun et al., "BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer," in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manage.*, Nov. 2019, pp. 1441–1450.
- [23] L. Ouyang et al., "Training language models to follow instructions with human feedback," in *Proc. NIPS*, Nov. 2022, pp. 27730–27744.
- [24] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," in *Proc. Int. Conf. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 4299–4307.
- [25] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.
- [26] G. Team et al., "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context," 2024, *arXiv:2403.05530*.
- [27] X. Bi et al., "DeepSeek LLM: Scaling open-source language models with longtermism," 2024, *arXiv:2401.02954*.
- [28] M. Jin et al., "Time-LLM: Time series forecasting by reprogramming large language models," in *Proc. ICLR*, Vienna, Austria, May 2024, pp. 1–24.
- [29] J. Ahn, R. Verma, R. Lou, D. Liu, R. Zhang, and W. Yin, "Large language models for mathematical reasoning: Progresses and challenges," 2024, *arXiv:2402.00157*.
- [30] B. Liu, X. Liu, S. Gao, X. Cheng, and L. Yang, "LLM4CP: Adapting large language models for channel prediction," *J. Commun. Inf. Netw.*, vol. 9, no. 2, pp. 113–125, Jun. 2024.
- [31] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Proc. NeurIPS*, Vancouver, BC, Canada, Dec. 2019, pp. 12360–12371.
- [32] D. Liu, P. Cheng, Z. Dong, X. He, W. Pan, and Z. Ming, "A general knowledge distillation framework for counterfactual recommendation via uniform data," in *Proc. 43rd Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2020, pp. 831–840.
- [33] N. Papernot, P. McDaniel, X. Wu, S. Jha, and A. Swami, "Distillation as a defense to adversarial perturbations against deep neural networks," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2016, pp. 582–597.
- [34] J. Gou, B. Yu, S. J. Maybank, and D. Tao, "Knowledge distillation: A survey," *Int. J. Comput. Vis.*, vol. 129, no. 6, pp. 1789–1819, 2021.
- [35] X. Xu et al., "A survey on knowledge distillation of large language models," 2024, *arXiv:2402.13116*.
- [36] G. E. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.
- [37] X. Wang, R. Zhang, Y. Sun, and J. Qi, "KDGAN: Knowledge distillation with generative adversarial networks," in *Proc. NeurIPS*, Dec. 2018, pp. 783–794.
- [38] S. Gupta, J. Hoffman, and J. Malik, "Cross modal distillation for supervision transfer," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 2827–2836.
- [39] N. Papernot, M. Abadi, Ú. Erlingsson, I. J. Goodfellow, and K. Talwar, "Semi-supervised knowledge transfer for deep learning from private training data," in *Proc. ICLR*, Toulon, France, Apr. 2017, pp. 1–14.
- [40] B. Lindauer. (Sep. 2020). *Insider Threat Test Dataset*. [Online]. Available: [https://kilthub.cmu.edu/articles/dataset/Insider\\_Threat\\_Test\\_Dataset/12841247](https://kilthub.cmu.edu/articles/dataset/Insider_Threat_Test_Dataset/12841247)
- [41] J. Glasser and B. Lindauer, "Bridging the gap: A pragmatic approach to generating insider threat data," in *Proc. IEEE Secur. Privacy Workshops*, May 2013, pp. 98–104.
- [42] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Min.*, Jun. 2008, pp. 413–422.
- [43] Q. Lin, H. Zhang, J.-G. Lou, Y. Zhang, and X. Chen, "Log clustering based problem identification for online service systems," in *Proc. IEEE/ACM 38th Int. Conf. Softw. Eng. Companion (ICSE-C)*, May 2016, pp. 102–111.
- [44] H. Guo, S. Yuan, and X. Wu, "LogBERT: Log anomaly detection via BERT," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2021, pp. 1–8.
- [45] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, vol. 1, Jun. 2019, pp. 4171–4186.
- [46] C. Zhang, X. Costa-Pérez, and P. Patras, "Adversarial attacks against deep learning-based network intrusion detection systems and defense mechanisms," *IEEE/ACM Trans. Netw.*, vol. 30, no. 3, pp. 1294–1311, Jun. 2022.





**Senming Yan** (Graduate Student Member, IEEE) received the B.E. degree in software engineering from Xidian University, China, in 2020. He is currently pursuing the joint Ph.D. degree with the Institute of Information Engineering, Chinese Academy of Sciences, and the School of Cyber Security, University of Chinese Academy of Sciences. His research interests include network security, zero trust security, and AI for security.



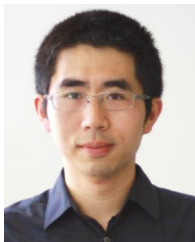
**Jing Ren** (Member, IEEE) received the B.E. and Ph.D. degrees in communication engineering from the University of Electronic Science and Technology of China (UESTC), Chengdu, China, in 2007 and 2015, respectively. She is currently an Associate Researcher with UESTC and a Research Assistant with the Peng Cheng Laboratory. Her research interests include network architecture and protocol design, software-defined networking, and network security.



**Lei Shi** (Graduate Student Member, IEEE) received the B.E. degree from the Electronic Information School, Wuhan University, China, in 2021. She is currently pursuing the Ph.D. degree with the School of Integrated Circuits (School of Electronic Information and Electrical Engineering), Shanghai Jiao Tong University. Her research interests include network security, intrusion detection, cybersecurity situational awareness, and AI for security.



**Ying Li** received the B.E. and Ph.D. degrees in communication engineering from the National University of Defense Technology, Changsha, China, in 2001 and 2006, respectively. She is currently a Researcher with the Peng Cheng Laboratory. Her research interests include network architecture design, zero trust security, and cognitive radio network.



**Wei Wang** (Senior Member, IEEE) received the B.S. degree from Central South University, Changsha, China, in 2010, the M.S. degree from Southeast University, Nanjing, China, in 2013, and the Ph.D. degree from The University of New South Wales, Sydney, Australia, in 2017. From 2018 to 2021, he was a Post-Doctoral Research Fellow with The University of New South Wales. From 2022 to 2025, he was a Senior Research Scientist with the Peng Cheng Laboratory, Shenzhen, China. Since 2026, he has been a Professor with the School of Information

Science and Technology, Harbin Institute of Technology, Shenzhen. His research interests include millimeter-wave communications, machine learning for wireless communications, and cybersecurity. He was a recipient of the 2023 IEEE ComSoc AP Outstanding Young Researcher Award and the Best Paper Awards at the IEEE ICC 2016 and 2024. He served as the co-chair for multiple symposia and workshops at major IEEE conferences. He serves as an Editor for IEEE TRANSACTIONS ON MOBILE COMPUTING and previously served as an Editor for IEEE WIRELESS COMMUNICATIONS LETTERS.



**Limin Sun** received the B.S. and Ph.D. degrees from the National University of Defense Technology, Changsha, China, in 1988 and 1998, respectively. He is currently a Professor with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China. He is also with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing. His research interests include mobile vehicle networks, the Internet of Things security, and wireless sensor networks.